GLOBAL ACADEMY OF TECHNOLOGY

(An Autonomous Institute under VTU)

Department of Computer Science & Engineering

Affiliated to VTU, Accredited by NAAC with 'A' grade

RR Nagar, Bengaluru – 560 098

(Accredited by NBA 2022-2025)

DESIGN AND ANALYSIS OF ALGORITHMS LABORATORY MANUAL

IV Semester

Course Code: CSEL23409

[As per the Choice Based Credit System Scheme]

Scheme: 2023

Version1.0

w.e.f 13th May 2025

Editorial Committee

DAA Lab Faculty, Dept. of CSE

Approved by

H.O.D, Department. of CSE

Document Log

Name of the document	Design and Analysis of Algorithms Laboratory Manual
Scheme	2023
Current version number and date	V 1.0 / 13.05.2025
Subject code	CSEL23409
Editorial Committee	Prof.Shwetha N Prof. Soumya B K Prof.Sowmya N
Computer Programmer	Ms. Preethi
Approved by	Dr. Kumaraswamy S, HOD, Dept. of CSE

Table of Contents

Sl. No.	Particulars		Page No.
1	Vision and Mission of Department		1
2	PEOs, PSOs, POs		2
3	• Course Outcomes • Syllabus • Conduction of Practical Examination • CO-PO-PSO Mapping		4
4	Lab Evaluation Process		8
5	Lab Rubrics		9
6	CHAPTER 1	BASICS OF ALGORITHMS	10
7	CHAPTER 2 : LAB SYLLABUS PROGRAMS		12
	Program 1	a) Implement Naïve String Matching Algorithm Using Brute Force Approach. b) Implement and Analyze and Iterative and recursive Binary Search algorithm Using Divide And Conquer Method.	12
	Program 2	Sort a given set of n integer elements using merge sort method and compute its time complexity. demonstrate this algorithm using divide and conquer method.	15
	Program 3	Sort a given set of n integers elements using quick sort method and compute its time complexity. demonstrate this algorithm using divide and conquer method.	18
	Program 4	a) Incorporate the array data structure and demonstrate whether a given unweighted graph is connected or not using dfs method . b) Implement the graph traversal technique using bfs method to print all the nodes reachable from a given starting node in an unweighted graph.	21
	Program 5	Demonstrate how to obtain the topological ordering of vertices in a given digraph.	25
	Program 6	Implement horspools string matching algorithm.	27
	Program 7	Compute the Transitive Closure for a given directed graph Using Warshalls Algorithm.	29
	Program 8	For a given weighted graph, construct an all-pairs shortest paths using floyd's algorithm.	30
	Program 9	Implement 0/1 knapsack problem using dynamic programming memory functions technique.	32
	Program 10	Find minimum cost spanning tree for a given weighted graph using prims and Kruskal's algorithm.	33
	Program 11	From a given vertex in a weighted connected graph, determine the single source shortest paths using dijkstra's algorithm.	38

	Program 12	**Demonstrate the working of back tracking approached for solving n-queen's problem.**	**40**
8	**CHAPTER 4**	VIVA QUESTIONS	**42**
9	**CHAPTER 5**	ADDITIONAL PROGRAMS	**46**
		APPENDIX - Time Complexity of Sorting Algorithms	**50**

Vision of the Department

To achieve academic excellence and strengthen the skills to meet emerging challenges of Computer Science and Engineering

Mission of the Department

M1: To impart strong theoretical foundations in the field of Computer Science and Engineering accompanied with extensive practical skills.

M2: To inculcate research and innovation spirit through interaction with industry and carry out projects that address societal needs.

M3: Instill professional ethics and values with concern for environment.

Program Educational Objectives (PEOs)

After the course completion, CSE graduates will be able to:

PEO1: Succeed in engineering / management positions with professional ethics.

PEO2: Engage in improving professional knowledge through certificate / post-graduate programs in engineering or management.

PEO3: Establish themselves as entrepreneurs and contribute to the Society.

Program Specific Outcomes (PSOs)

PSO1: Design, Implement and Test System Software and Application Software to meet the desired needs.

PSO2: Develop solutions in the area of Communication Networks, Database Systems and Computing Systems.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one‘s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Details

Course Name : Design and Analysis of Algorithms Laboratory
Course Code : CSE23409
Course prerequisite : Basics of Programming in C, Data Structures.

Course Objectives

CLO1: Apply mathematical concepts and notations to define a problem.

CLO2: Understand and apply algorithms design techniques.

CLO3: Gain ability to solve real life problems using algorithms techniques.

CLO4: Understand the limitations of Algorithmic power.

COURSE OUTCOMES

Upon successful completion of this course, student will be able to:

COs	COURSE OUTCOMES
CO409.1	Explain the basic techniques of analyzing the algorithms using time & space complexity and asymptotic notations.
CO409.2	Devise algorithms using brute force and Divide and Conquer techniques for a given problem.
CO409.3	Demonstrate Graph Algorithms using greedy method, Transform and Conquer Approach to model Engineering Problems.
CO409.4	Employ Dynamic Programming and Decrease & Conquer strategies to solve a given problem.
CO409.5	Use Back Tracking, Branch and Bound design techniques for solving Computationally hard problems.

Syllabus

Course Code	**CSEL23409**	**CIE Marks**	**50**
Hours/Week (L: T: P)	**0:0:2**	**SEE Marks**	**50**
No. of Credits	**1**	**Examination Hours**	**03**

Laboratory Experiments

Implement the specified algorithms for the following problems and compute its time complexity under LINUX/Windows environment.

Program 1	**a) Implement Naïve String Matching Algorithm Using Brute Force Approach.** **b) Implement and Analyze and Iterative and recursive Binary Search algorithm Using Divide And Conquer Method.**
Program 2	**Sort a given set of n integer elements using merge sort method and compute its time complexity. demonstrate this algorithm using divide and conquer method.**
Program 3	**Sort a given set of n integers elements using quick sort method and compute its time complexity. demonstrate this algorithm using divide and conquer method.**
Program 4	**a) Incorporate the array data structure and demonstrate whether a given unweighted graph is connected or not using dfs method .** **b) Implement the graph traversal technique using bfs method to print all the nodes reachable from a given starting node in an unweighted graph.**
Program 5	**Demonstrate how to obtain the topological ordering of vertices in a given digraph.**
Program 6	**Implement horspools string matching algorithm.**
Program 7	**Compute the Transitive Closure for a given directed graph Using Warshalls Algorithm.**
Program 8	**For a given weighted graph, construct an all-pairs shortest paths using floyd's algorithm.**
Program 9	**Implement 0/1 knapsack problem using dynamic programming memory functions technique.**
Program 10	**Find minimum cost spanning tree for a given weighted graph using prims and Kruskal's algorithm.**
Program 11	**From a given vertex in a weighted connected graph, determine the single source shortest paths using dijkstra's algorithm.**
Program 12	**Demonstrate the working of back tracking approached for solving n-queen's problem.**

Conduction of Practical Examination:

1. All laboratory programs are to be included for practical examination.
2. Students are allowed to pick one program from lot.
3. Strictly follow the instructions as printed on the cover page of answer script.
4. Marks distribution:

Procedure + Conduction + Viva: **15 + 70 +15 = 100 Marks(Scale downed to 10)**

5. **Change of experiment is allowed only once and marks allotted to the procedure part to be made zero.**

CO-PO-PSO Mapping

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CSEL23409.1	3	3	3	-	3	-	-	-	-	-	-	2	3	-
CSEL23409.2	3	3	3	-	3	-	-	-	-	-	-	2	3	-
CSEL23409.3	3	3	3	-	3	-	-	-	-	-	-	2	3	-
CSEL23409.4	3	3	3	-	3	-	-	-	-	-	-	2	3	-
Avg	**3**	**3**	**3**	**-**	**3**	**-**	**-**	**-**	**-**	**-**	**-**	**2**	**3**	**-**

LAB EVALUATION PROCESS WEEK WISE

EVALUATION OF EACH PROGRAM

S. No	ACTIVITY	MARKS
1	Record +Viva	**10**

INTERNAL ASSESSMENT EVALUATION
(End of Semester)

S. No	ACTIVITY	MARKS
1	Write-up	15
2	Conduction	70
3	Viva	15
	TOTAL	100(Scaled down to 10)

FINAL INTERNAL ASSESSMENT CALCULATION		
S. No	**ACTIVITY**	**MARKS**
1	Average of weekly entries	10
2	Internal Assessment	10
	TOTAL	**20**

LAB RUBRICS

Rubrics for Evaluation of Lab Record:

Attribute	Max Marks	Good	Satisfactory	Poor
		4-5	1-3	0
Write Up and Time Management	5	• Written complete program without any errors • Written input and expected output for all the test cases. • Written program with proper indentation • Index entry promptly done and submits on time	• Written Program with few logical errors. • Written input and expected output for few test cases. • Written program with moderate indentation	• Not submitted • Input and expected output is not written
	Max Marks	15-20	5-14	0-4
Conduction of experiment	20	• All possible cases of input and output demonstrated	• The code is not tested with all possible cases.	• Unable to execute within the allotted
	Max Marks	4-5	1-3	0
Viva	5	• Able to explain the design of algorithm and logic. • Able to answer all question related to the concept.	• Partially understood the logic of the experiment. • Able to answer 1 out of 3 questions	• Not understood the logic of the experiment. • Not answering any question

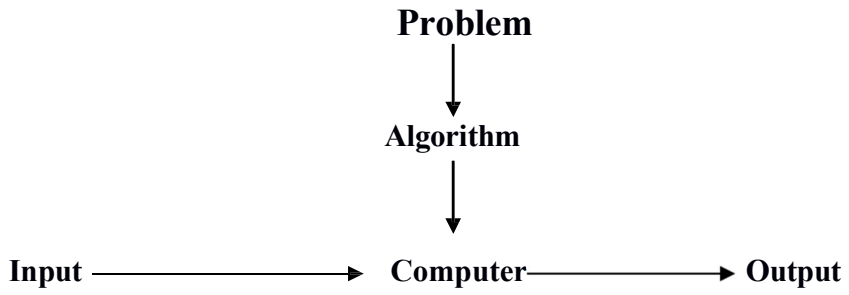
Rubrics for Evaluation of Internal Test:

Attribute	Max Marks	Excellent	Good	Satisfactory	Poor
		11-15	6-10	1-5	0
Write-up	15	• Completes source code. • No syntax and logical errors • All possible inputs listed with expected output	• Completes source code. • Syntax and logical errors exist.	• Incomplete source code. • All possible inputs listed with expected output	• Change of program
	Max Marks	60-70	20-59	11-19	0-10
Execution	70	• Able to debug the program and get the right set of outputs • Able to modify the Program as per the examiner's Direction	• Able to execute program for few possible inputs without errors	• Able to clear the errors. • Not able to execute program	• Not able to clear the errors and execute the program
	Max Marks	11-15	6-10	1-5	0
Viva - Voce	15	• Able to answer all questions correctly	• Able to answer 3 questions out of 5 questions correctly	• Able to answer 2 - 5 questions	• Not able to answer any questions.

CHAPTER 1

BASICS OF ALGORITHMS

An algorithm is a sequence of finite number of unambiguous instructions for solving a problem, i.e for obtaining a required output for any legitimate input in a finite amount of time.



- The non-ambiguity requirement for each step of an algorithm.
- The range of inputs has to be specified carefully.
- The same algorithm can be represented in several different ways.
- Several algorithms for solving the same problem may exist.
- Algorithm for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

Properties Of Algorithms

1. **Input** – an algorithm has zero or more inputs.
2. **Output** – an algorithm has one or more outputs.
3. **Finiteness** – an algorithm must terminate after a finite number of steps.
4. **Definiteness** – each step must be precisely defined.
5. **Effectiveness** – all operations can be carried out exactly in finite time (lesser amount of time).

Algorithm Paradigms

Often there are large collections of problems that can be solved using the same general techniques or paradigms. A few of the most common are described below:

Brute Force: A straightforward approach to solving a problem based on the problem statement and concepts involved. Brute force algorithms are rarely efficient.

Examples:

- Bubble sort.
- Computing the sum of n numbers by direct addition.
- Standard matrix multiplication.
- Linear search.

Divide And Conquer: Perhaps the most famous algorithm paradigm, divide and conquer is based on partitioning the problem into two or more smaller sub-problems, solving them and combining the sub-problem solutions into a solution for the original problem.

Examples:

- Binary Search, Merge sort and quick sort.
- The Fast Fourier Transform (FFT).
- Strassen's matrix multiplication.

Greedy Algorithms: Greedy algorithms always make the choice that seems best at the moment. This is locally optimal choice is made with the hope that it leads to a globally optimal solution. Some greedy algorithms may not be guaranteed to always produce an optimal solution. Greedy algorithms are often applied to combinatorial optimization problems. Given an instance of the problems, there is a set of candidates or feasible solutions that satisfy the constraints of the problem. For each feasible solution there is a value determined by an objective function. An optimal solution minimizes (or maximizes) the value of objective function.

Examples:

- Kruskal's and Prim's minimal spanning tree algorithms.
- Dijkstra‟s single source shortest path algorithm.
- Huffman coding.

Dynamic Programming: A nutshell definition of dynamic programming is difficult, but to summarize, problems which lend themselves to a dynamic programming attack have the following characteristics: We have to search over a large space for an optimal solution. The optimal solution can be expressed in terms of optimal solution to sub-problem. The number of sub-problems that must be solved is small. Dynamic programming algorithms have the following features:

- A recurrence that is implemented iteratively.
- A table, built to support the iteration.
- Tracing through the table to find the optimal solution.
- Examples - Fibonacci number computation, Knapsack problems, Floyd's and Warshall's algorithm.

CHAPTER 2

LAB SYLLABUS PROGRAMS

Program 1.

a. . Implement naïve String matching algorithm using Brute Force Approach.

ABOUT THE EXPERIMENT

Naïve String Matching Algorithm (often referred to as the brute force approach) is a simple and intuitive algorithm used to find a pattern in a given text. The basic idea is to check every possible position in the text where the pattern might start and then compare the pattern with the substring of the text. This experiment focuses on understanding how the Naïve String Matching Algorithm works in practice, analyzing its performance, and highlighting its inefficiencies.

ADVANTAGES

- Simplicity: The algorithm is simple to understand and implement, making it a great introductory topic for learning about string matching.
- No Preprocessing: The algorithm doesn't require any preprocessing or auxiliary data structures, unlike more complex algorithms like KMP or Boyer-Moore.
- Works for Small Inputs: For smaller strings and patterns, the Naïve String Matching Algorithm is often sufficient and works well.

Source Code

```
#include <stdio.h>
#include <string.h>

// Function to implement the Naive String Matching Algorithm (Brute Force Approach)
void naiveStringMatch(char text[], char pattern[])
 {
   int n = strlen(text);     // Length of the text
   int m = strlen(pattern);  // Length of the pattern

   // Traverse the text
   for (int i = 0; i <= n - m; i++)
{
      int j = 0;

      // Check for the pattern match at position i
      while (j < m && text[i + j] == pattern[j]) {
         j++;
      }

        // If the full pattern is matched
      if (j == m)
{
         printf("Pattern found at index %d\n", i);
```

```
        }
    }
}

int main()
{
    char text[] = "ABABDABACDABABCABAB";
    char pattern[] = "ABAB";

    // Call the function to find pattern in text
    naiveStringMatch(text, pattern);

    return 0;
}
```

OUTPUT:

Execution of Program:

```
cc string.c
./a.out

Pattern found at index 0
Pattern found at index 10
Pattern found at index 15

=== Code Execution Successful ===
```

b. Implement and analyze iterative and recursive binary search algorithm using divide and conquer method.

Applications

- Dictionary Lookups:Quickly finding a specific word in a sorted dictionary, like in a spell checker or autocorrect feature.
- Database Indexing:Locating relevant rows in a database by searching through sorted indexes, especially in B-trees or B+ trees.
- Navigation Systems:Finding the closest location to a user's current position on a sorted map.
- Game Development:Efficiently identifying collisions between objects in a game by checking against sorted data structures
- Data Analysis:Finding the closest value to a target within a sorted dataset, particularly in numerical analysis
- Text Editing:Implementing "find" functionality in a text editor where the text is sorted alphabetically .

Sourcse Code:

```
#include <stdio.h>
int iterativeBinarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;

    // Repeat while the left index is less than or equal to the right index
    while (left <= right) {
        int mid = left + (right - left) / 2;

        // If the target is found at mid
        if (arr[mid] == target) {
            return mid; // Return the index
        }

        // If the target is smaller, ignore the right half
        if (arr[mid] > target) {
            right = mid - 1;
        }
        // If the target is larger, ignore the left half
        else {
            left = mid + 1;
        }
    }

    // Target not found
    return -1;
}

int main() {
    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15, 17};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 7;

    int result = iterativeBinarySearch(arr, n, target);

    if (result != -1)
  {
        printf("Target %d found at index %d\n", target, result);
    } else {
        printf("Target %d not found in the array\n", target);
    }
    return 0;
}
```

OUTPUT:

Execution of Program:

```
cc string.c
./a.out
Target 7 found at index 3
=== Code Execution Successful ===
```

Program 2

Sort a given set of n integer elements using Merge Sort method and compute its time complexity.

ABOUT THE EXPERIMENT:

Merge sort is a perfect example of a successful application of the divide-and-conquer technique. The steps followed in merge sort are:

- Merge sort works by dividing the given array into two sub arrays of equal size
- The sub arrays are sorted independently using recursion
- The sorted sub array are then merged to get the solution for the original array
- In merge sort given set of elements (n) split into sets a[1]……a[n/2] and a[n/2+1]……a[n].
- Each set is individually sorted and resulting sorted sequence merged to produce a single sorted sequence of n elements

Time Complexity Analysis:

Best/Average/Worst case performance is Θ (n log n)

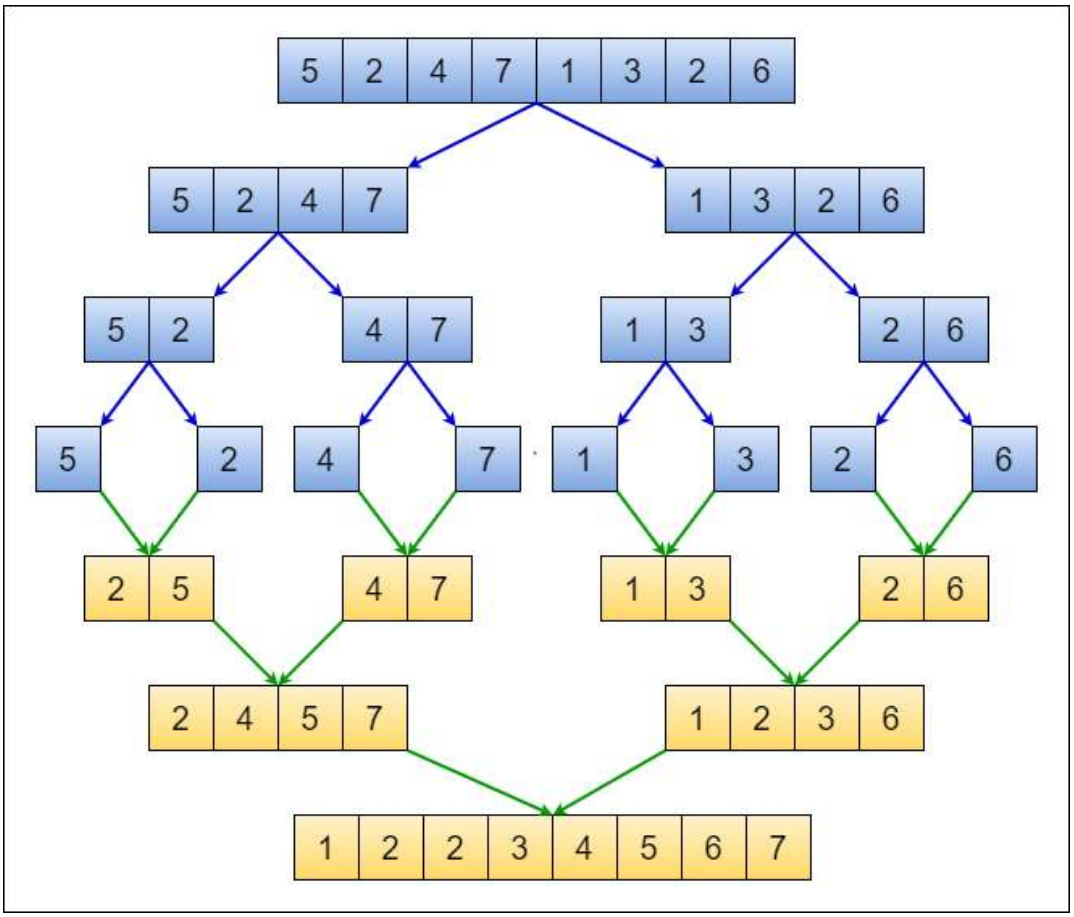


Fig 1:Tree Construction for dividing the array and Tree construction for Merging sorted array.

APPLICATIONS:

- Merge sort is the best algorithm to sort a linked list with the complexity of O(n logn).

- Merge sort helps to solve this problem by telling the number of inversion pairs in an unsorted array. The inversion count problem tells how many pairs need to be swapped in order to get a sorted array. Merge sort works best for solving this problem.
- **External sorting:** Merge sort is an external sorting technique. Thus, if we have data of 1GB but the available RAM size is 500MB, then we will use merge sort.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>

int count;

void merge(int a[10], int left, int mid, int right)
{
    int i, j, k, b[10];

    i = left;
    j = mid+1;
    k = left;
    while( (i<=mid) && (j<=right))
    {
       count++;
       if(a[i] < a[j])
        {
                b[k++] = a[i++];
        }
       else
        {
                b[k++] = a[j++];
        }
    }
    while(i <= mid)
    b[k++] = a[i++];
    while(j <= right)
       b[k++] = a[j++];
   for(i=left; i<=right; i++)
       a[i] = b[i];
}
void mergesort(int a[10], int left, int right)
{
    int mid;
    if(left < right)
    {
       mid = (left + right)/2;
       mergesort(a,left,mid);
       mergesort(a,mid+1,right);
       merge(a,left,mid,right);
    }
      }
```

```
int main()
{
    int i,j,n,a[10];
    printf("\n Enter no of
    elements: "); scanf("%d",&n);
    printf("\nEnter
    elements\n");
    for(i=0;i<n;i++)
       scanf("%d",&a[i]);
    mergesort(a,0,n-1);
    printf("\n Sorted elements:
    \n"); for(i=0;i<n;i++)
       printf("%d\n",a[i]);
   printf("\n Number of counts :
    %d\n",count); return 0;
}
```

OUTPUT:

Execution of the Program:

cc merge.c
./a.out

OUTPUT:

RUN1	RUN2	RUN3
INPUT:	**INPUT:**	**INPUT:**

Program 3

Sort a given set of n integer elements using Quick Sort method and compute its time complexity.

ABOUT THE EXPERIMENT:

Quick sort is one of the sorting algorithms working on the Divide and Conquer strategy. In Quick sort the given array elements is divided into two parts based on pivot (a key element) . Pivot element partitions the array into three sub arrays such that

- The left sub array contains all elements which are less than or equal to the pivot
- The middle sub arrays contains pivot
- The right sub array contains all elements which are greater than or equal to the pivot. The innermost loop is so efficient that it runs faster than mergesort and heapsort.

Time Complexity Analysis: Quick Sort
Algorithm Best case performance is **Ω(n log)**
Average case performance is **Θ(1.38*nlog n)**
Worst case Performance is **O(n^2)**

APPLICATIONS:

- The sorting algorithm is used for **information searching** and as Quicksort is the fastest algorithm so it is widely used as a better way of searching.
- It is used everywhere where a **stable sort** is not needed.
- Quicksort is a cache friendly algorithm as it has a good locality of reference when used for arrays.
- It is **tail -recursive** and hence all the call optimization can be done.
- It is an in-place sort that does not require any extra storage memory.
- It is used in operational research and event-driven simulation.
- Numerical computations and in scientific research, for accuracy in calculations most of the efficiently developed algorithm uses priority queue and quick sort is used for sorting.
- Variants of Quicksort are used to separate the **Kth smallest** or **largest elements**.

Source Code:

```
#include  <stdio.h>
#include <stdlib.h>
int count;

int partition(int a[10], int low, int high)
{
    int i,j,pivot,temp;
    i = low + 1;
    j = high;
    pivot = a[low];
    while(1)
    {
            while ((pivot >= a[i]) && i <= high)
            {
                    i++;
                    count++;
             }
             count++;
            while(pivot < a[j] && j>=low)
            {
                    j--;
                    count++;
            }
             count++;
            if(i < j)
            {
                    temp = a[i];
                    a[i] = a[j];
                    a[j] = temp;
             }
            else
             {
                    temp = a[low];
                    a[low] = a[j];
                    a[j] = temp;
                    return j;
             }
    }

}

void quicksort(int a[10],int low,int high)
{
    int i,j;
    if(low < high)
    {
            j = partition(a, low, high);
```

```
            quicksort(a, low, j-1);
            quicksort(a, j+1, high);
        }
    }

    int main()
    {
        int a[10];
        int i,n;
        printf("\nEnter no of elements: ");
        scanf("%d",&n);

        printf("\nEnter elements \n");
        for(i=1;i<=n;i++)
             scanf("%d",&a[i]);
        quicksort(a,1,n);

        printf("Sorted elements: \n");
        for(i=1;i<=n;i++)
            printf("%d\t",a[i]);
        printf("\n Number of counts : %d\n",count);
        return 0;

    }
```

OUTPUT:

Execution of the Program:

cc quick.c
./a.out

OUTPUT: (Plot a Graph for time complexity verses input size)

RUN1	RUN2	RUN3
INPUT:	**INPUT:**	**INPUT:**

Program 4

a) Incorporate the array data structure and demonstrate whether a given unweighted graph is connected or not using dfs method .

APPLICATIONS:

- For an unweighted graph, DFS traversal of the graph produces the minimum spanning tree.
- Detecting cycle in a graph
- Path Finding
- Topological Sorting
- To test if a graph is bipartite
- Finding Strongly Connected Components of a graph
- Solving puzzles with only one solution, such as mazes.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>

int topo[10],k;

void dfs(int a[10][10],int n,int v[10],int source)
{
  int i;
  v[source]=1;
  for(i=1;i<=n;i++)
  {
     if(v[i]==0 && a[source][i]==1)
       dfs(a,n,v,i);
  }
  topo[++k]=source;
}

int main()
{
   int n,i,j,a[10][10],v[10];
   printf("\n Enter the no of Vertices : ");
   scanf("%d",&n);
   printf("\n Enter the Adjacency matrix\n");
   for(i=1;i<=n;i++)
          for(j=1;j<=n;j++)
                    scanf("%d",&a[i][j]);
   for(i=1;i<=n;i++)
    v[i]=0;
   for(i=1;i<=n;i++)
   {
      if(v[i]==0)
       dfs(a,n,v,i);
```

```
        }
        printf("\n The topological ordering is\n");
        for(i=k;i>=1;i--)
                printf("%d\t",topo[i]);
    }
```

OUTPUT:

```
Enter the number of vertices: 3
Enter the adjacency matrix of the graph:
0 0 0
1 0 1
1 1 0
The graph is not connected.
```

```
Enter the number of vertices: 3
Enter the adjacency matrix of the graph:
0 1 1
1 0 1
1 1 0
The graph is connected.
```

c) Implement the graph traversal technique using bfs method to print all the nodes reachable from a given starting node in an unweighted graph.

```
 #include <stdio.h>
#include <stdlib.h>
#define MAX 100 // Maximum number of vertices in the graph
// Function prototypes
void BFS(int graph[MAX][MAX], int n, int start);
int main() {
    int n, i, j, start;
    int graph[MAX][MAX]; // Adjacency matrix representation of the graph
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix of the graph:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
   printf("Enter the starting vertex: ");
    scanf("%d", &start);
    printf("Nodes reachable from vertex %d using BFS:\n", start);
    BFS(graph, n, start);

    return 0;
}

// Breadth-First Search function
void BFS(int graph[MAX][MAX], int n, int start) {
    int visited[MAX] = {0}; // Array to track visited vertices
    int queue[MAX];         // Queue for BFS traversal
    int front = 0, rear = 0;
    // Mark the starting vertex as visited and enqueue it
    visited[start] = 1;
    queue[rear++] = start;
    while (front < rear) {
        // Dequeue a vertex from the queue
        int current = queue[front++];
        printf("%d ", current); // Print the current vertex
        // Enqueue all unvisited neighbors of the current vertex
        for (int i = 0; i < n; i++) {
            if (graph[current][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
```

```
        }
      }
    }
  }
```

OUTPUT:

Connected Graph

Input:

```text
Enter the number of vertices: 5
Enter the adjacency matrix of the graph:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter the starting vertex: 0
```

Output:

```text
Nodes reachable from vertex 0 using BFS:
0 1 2 3 4
```

Unconnected Graph

Input:

```text
Enter the number of vertices: 4
Enter the adjacency matrix of the graph:
0 1 0 0
1 0 0 0
0 0 0 1
0 0 1 0
Enter the starting vertex: 2
```

Output:

```text
Nodes reachable from vertex 2 using BFS:
2 3
```

Program5.

Demonstrate how to obtain the topological ordering of vertices in a given digraph..

```
#include <stdio.h>
#define MAX 10  // Maximum number of vertices
int adj[MAX][MAX];   // Adjacency matrix
int visited[MAX];    // Visited array
int stack[MAX];      // Stack for topological sorting
int top = -1;        // Stack top index
// Function to push to stack
void push(int v) {
    top++;
    stack[top] = v;
}
// Function to perform DFS
void dfs(int v, int n) {
    visited[v] = 1; // Mark as visited

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && visited[i] == 0) {
            dfs(i, n);
        }
    }

    push(v); // Push to stack after all adjacent nodes are visited
}


// Function to perform Topological Sorting using DFS
void topologicalSort(int n) {
    for (int i = 0; i < n; i++) {
        visited[i] = 0; // Initialize visited array
    }

    // Call DFS for each unvisited node
    for (int i = 0; i < n; i++) {
        if (visited[i] == 0) {
            dfs(i, n);
        }
    }

    // Print the stack (Topological Order)
    printf("Topological Order: ");
    while (top >= 0) {
        printf("%d ", stack[top]);
        top--;
    }
    printf("\n");
}
```

```
// Driver Code
int main() {
    int n, edges, u, v;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Initialize adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
    }

    printf("Enter edges (format: src dest):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1; // Directed edge from u to v
    }

    topologicalSort(n);

    return 0;
}
```

Input

Enter the number of vertices: 6
Enter the number of edges: 6
Enter edges (format: src dest):
5 2
5 0
4 0
4 1
2 3
3 1

Output

Topological Order: 5 4 2 3 1 0

Program 6

Implement horspools string matching algorithm.

```
#include <stdio.h>
#include <string.h>
#define MAX 256 // Maximum size for the alphabet
// Function to create the shift table
void createShiftTable(char *pattern, int patternLength, int shiftTable[]) {
    for (int i = 0; i < MAX; i++) {
        shiftTable[i] = patternLength; // Default shift value is the length of the pattern
    }
    for (int i = 0; i < patternLength - 1; i++) {
        shiftTable[(unsigned char)pattern[i]] = patternLength - 1 - i;
    }
}
// Function to implement Horspool's string matching algorithm
int horspoolMatching(char *text, char *pattern) {
    int textLength = strlen(text);
    int patternLength = strlen(pattern);
    int shiftTable[MAX];
    createShiftTable(pattern, patternLength, shiftTable);
    int i = patternLength - 1; // Start with the end of the first window
    while (i < textLength) {
        int k = 0;
        // Compare the pattern from right to left
        while (k < patternLength && pattern[patternLength - 1 - k] == text[i - k]) {
            k++;
        }
        if (k == patternLength) { // Match found
            return i - patternLength + 1;
        } else { // Shift based on the shift table
            i += shiftTable[(unsigned char)text[i]];
        }

    return -1; // No match found
}
int main() {
    char text[100], pattern[50];
    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = '\0'; // Remove newline character
    printf("Enter the pattern: ");
    fgets(pattern, sizeof(pattern), stdin);
    pattern[strcspn(pattern, "\n")] = '\0'; // Remove newline character
    int position = horspoolMatching(text, pattern);
```

```
    if (position != -1) {
        printf("Pattern found at position: %d\n", position);
    } else {
        printf("Pattern not found in the text.\n");
    }
    return 0;
}
```

OUTPUT:

Input:

```text
Enter the text: hello world
Enter the pattern: world
```

Output:

```text
Pattern found at position: 6
```

Program 7.

Compute the Transitive Closure for a given directed graph Using Warshalls Algorithm.

```
#include<stdio.h>
void warsh(int p[][10],int n)
{
 int i,j,k;
 for(k=1;k<=n;k++)
 for(i=1;i<=n;i++)
 for(j=1;j<=n;j++)
 p[i][j]=p[i][j] || p[i][k] && p[k][j];
}
int main()
{
 int a[10][10],n,i,j;
printf("\nEnter the n value:");
 scanf("%d",&n);
 printf("\nEnter the graph data:\n");
 for(i=1;i<=n;i++)
 for(j=1;j<=n;j++)
 scanf("%d",&a[i][j]);
 warsh(a,n);
 printf("\nResultant path matrix\n");
 for(i=1;i<=n;i++)
 {
  for(j=1;j<=n;j++)
  printf("%d ",a[i][j]);
  printf("\n");
 }
 return 0;
}
```

Output:

```
Enter the n value:3

Enter the graph data:
1 0 1
1 1 1
0 1 1

Resultant path matrix
1 1 1
1 1 1
1 1 1
```

Program 8.

For a given weighted graph, construct an all-pairs shortest paths using floyd's algorithm.

```
#include<stdio.h>
#include<conio.h>
#define INF 999
int min(int a,int b)
{
 return(a<b)?a:b;
}
void floyd(int p[][10],int n)
{
 int i,j,k;
 for(k=1;k<=n;k++)
 for(i=1;i<=n;i++)
 for(j=1;j<=n;j++)
 p[i][j]=min(p[i][j],p[i][k]+p[k][j]);
}
void main()
{
 int a[10][10],n,i,j;

printf("\nEnter the n value:");
 scanf("%d",&n);
 printf("\nEnter the graph data:\n");
 for(i=1;i<=n;i++)
 for(j=1;j<=n;j++)
 scanf("%d",&a[i][j]);
 floyd(a,n);
 printf("\nShortest path matrix\n");
 for(i=1;i<=n;i++)
 {
  for(j=1;j<=n;j++)
  printf("%d ",a[i][j]);
  printf("\n");
 }
 getch();
}
```

Output:

```
Enter the n value:3

Enter the graph data:
0 999 1
3 0 999
2 1 0

Shortest path matrix
0 2 1
3 0 4
2 1 0
```

Program 9

Implement 0/1 knapsack problem using dynamic programming memory functions technique.

```
#include<stdio.h>
int w[10],p[10],n;
 int max(int a,int b)
{
 return a>b?a:b;
}
int knap(int i,int m)
{
 if(i==n) return w[i]>m?0:p[i];
 if(w[i]>m) return knap(i+1,m);
 return max(knap(i+1,m),knap(i+1,m-w[i])+p[i]);
}
int main()
{
 int m,i,max_profit;
 printf("\nEnter the no. of objects:");
 scanf("%d",&n);
 printf("\nEnter the knapsack capacity:");
 scanf("%d",&m);
 printf("\nEnter profit followed by weight:\n");
 for(i=1;i<=n;i++)
 scanf("%d %d",&p[i],&w[i]);
 max_profit=knap(1,m);
 printf("\nMax profit=%d",max_profit);
 return 0;
}
```

Output:

```
Enter the no. of objects: 4
Enter the knapsack capacity: 5
Enter profit followed by weight:
25 3
26 2
19 1
28 4
Max profit = 51
```

Program 10:

Find minimum cost spanning tree for a given weighted graph using prims and kruskals algorithm.

Prims:

```
#include <stdio.h>
#include <stdlib.h>
void prims(int cost[10][10],int n)
{
 int a,b,i,j,visited[10],mincost=0,u,v,min,ne=1;
 for(i=1;i<=n;i++)
 visited[i]=0;
 printf("\n The edges of the minimum spanning tree \n ");
 visited[1]=1;
 while(ne<n)
 {
 for(i=1,min=999;i<=n;i++)
 {
 for(j=1;j<=n;j++)
 {
 if(cost[i][j] < min)
 {
if (visited[i]== 1)
 continue;
 else
 {
 min=cost[i][j];
 a=u=i;
 b=v=j;
 }
 }
 }
 }
 if(visited[u] == 0 || visited[v] == 0)
 {
 printf("\n %d Edge(%d,%d) = %d",ne++,a,b,min);
 mincost = mincost+min;
visited[b]=1;
 }
Cost[a][b]=cost[b][a]=999;
}
 Printf("\n Minimum total cost = %d\n",mincost);
}
Int main()
    {
```

```
Int n,I,j,cost[10][10];
Printf("\n Read number of nodes :");
Scanf("%d",&n);
Printf("\n read cost matrix\n");
For(i=1;i<=n;i++)
{
For(j=1;j<=n;j++)
{
Scanf("%d",&cost[i][j]);
If(cost[i][j] == 0)
Cost[i][j]=999;
}
}
Prims(cost,n);
Return 0;

}
```

OUTPUT:

RUN	RUN
INPU	**INPU**

b) Kruskal's:

```
#include<stdio.h>
#define INFINITY 999
#define MAX 10
struct EDGE
{
 int x, y, wt;
}
e[MAX];
int parent[MAX];
int cost[MAX][MAX];
int t[MAX][2];
int nedges;
int eno;
void sort_edges()
{
 int i,j;
 struct EDGE temp;
 for(i=1; i < nedges; i++)
 for(j=1; j < nedges-i; j++)
 if(e[j].wt > e[j+1].wt)
 {
temp = e[j];
e[j] = e[j+1];
e[j+1] = temp;
 }
}
int get_parent(int v)
{
while(parent[v])

v = parent[v];
return v;
}
void join(int i, int j)
{
parent[j] = i;
}
void kruskal(int n)
{
int i,j,k,sum=0;
int eno = 1;
struct EDGE nextedge;
for(k=1; k < n; )
```

```
{
nextedge = e[eno++];
i = get_parent( nextedge.x );
j = get_parent( nextedge.y );
if(i != j)
{
join(nextedge.x, j); //e. parent[j] = nextedge.x;
t[k][1] = nextedge.x;
t[k][2] = nextedge.y;
sum = sum + nextedge.wt;
k++;
    }
    }
    printf("\nCost of the spanning tree is: %d\n",sum);
    printf("\nThe edges of the spanning tree are:\n");
    for(i=1;i<n;i++)
    printf("%d -> %d\n",t[i][1],t[i][2]);
    }
    int main()
    {
    int i,j;
    int n;
    printf("\nEnter the no.of vertices: ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    parent[i] = 0;
    eno = 1;
    printf("\nEnter the cost adjacency matrix: 0 = self loop & 999 = no edge\n");
    for(i=1;i<=n;i++)
    {
    for(j=1;j<=n;j++)
     {
    scanf("%d",&cost[i][j]);
    if(i == j || cost[i][j] == INFINITY)
    continue;
    e[eno].x = i;
    e[eno].y = j;
    e[eno].wt = cost[i][j];
    eno++;
    nedges++;
     }
    }
    sort_edges();
    kruskal(n);
```

```
return 0;
}
```

OUTPUT:

RUN	RUN
INPUT	**INPUT**

Program 11.

From a given vertex in a weighted connected graph, determine the single source shortest paths using dijkstra's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#define INFINITY 999
void dijkstra(int cost[10][10],int n,int source,int v[10],int d[10])
{
 int min,i,j,u;
 v[source] = 1;
 for(i=1; i<=n; i++)
 {
 min = INFINITY;
 for(j=1; j<=n; j++)
 {
 if(v[j] == 0 && d[j] < min)
 {
 min = d[j];
 u = j;
 }
 }
 v[u] = 1;
 for(j=1; j<=n; j++)
 {
 if(v[j] == 0 && (d[j] > (d[u] + cost[u][j])))
 d[j] = d[u] + cost[u][j];
 }
 }
}
int main()
{
 int n,cost[10][10],sourceNode,v[10],d[10];
 int i, j;
printf("Enter no of vertices: ");
 scanf("%d",&n);
 printf("Enter Cost Adjacency matrix:\n");
 for(i=1; i<=n; i++)
 for(j=1; j<=n; j++)
 scanf("%d",&cost[i][j]);
 printf("Enter the source (%d to %d):\n",1,n);
 scanf("%d",&sourceNode);
 for(i=1; i<=n; i++)
 {
 d[i] = cost[sourceNode][i];
```

```
    v[i] = 0;
    }
    dijkstra(cost, n, sourceNode, v, d);
    printf("Shortest distance from %d\n\n",sourceNode);
    for(i=1; i<=n; i++)
    printf("%d --> %d = %d\n\n",sourceNode,i,d[i]);
    return 0;
   }
```

OUTPUT:

RUN	RUN
INPU	**INPU**

Program 12:

Demonstrate the working of back tracking approached for solving n-queen's problem.

```
#include<stdio.h>
#include<stdlib.h>
int x[10];
static int c=1;
void print Board(int n)
{
 int i,j;
 print("Solution %d:\n",c++);
 for(i=1;i<=n;i++)
 {
 for(j=1;j<=n;j++)
 {
 if(x[i]==j)
 printf("Q\t");
 else
 printf("-\t");
 }
 printf("\n");
 }
}
int place(int k, int i)
{
 int j;
 for(j=1;j<k;j++)
 {
 if((x[j]==i) || abs(x[j]-i)==abs(j-k))
 return 0;
 }
 return 1;
}
void nQueens(int k, int n)
{
 int i,j;
 for(i=1;i<=n;i++)
 {
 if(place(k,i))
 {
 x[k]=i;
 if(k==n)
 {
 printf("\n");
printBoard(n);
}
else
nQueens(k+1,n);
}
}
}
int main()
```

```
{
 int n;
 printf("Enter the number of queens:\n");
 scanf("%d",&n);
 nQueens(1,n);
 if(c==1)
 printf("No solutions!");
}
```

OUTPUT:

RUN INPU	RUN INPU

CHAPTER 4

VIVA QUESTIONS AND ANSWERS

1. **What is an algorithm?**
   An algorithm is a sequence of finite number of unambiguous instructions for solving a problem. i.e., for obtaining a required output for any legitimate input in a finite amount of time.

2. **Name some basic Efficiency classes.**
   Constant, Logarithmic, Linear, nlogn, Quadratic, Cubic, Exponential, Factorial

3. **What are algorithm design techniques?**
   Algorithm design techniques (or strategies or paradigms) are general approaches to solving problems algorithmically, applicable to a variety of problems from different areas of computing. General design techniques are:
   (i) Brute force (ii) divide and conquer (iii) Decrease and conquer (iv) transform and conquer (v) Greedy technique (vi) dynamic programming (vii) Backtracking (viii) branch and bound

4. **How is the efficiency of the algorithm defined?**
   The efficiency of an algorithm is defined with the components.
   - Time efficiency -indicates how fast the algorithm runs
   - Space efficiency -indicates how much extra memory the algorithm needs

5. **Write the Analysis for the Quicksort.**
   O(nlogn) in average and best cases $O(n^2)$ in worst case

6. **Is insertion sort better than the mergesort?**
   Insertionsortworksexceedinglyfastonarraysoflessthen16elements,though for large,, n‟ its computing time is $O(n^2)$.

7. **Specify the algorithms used for constructing Minimum cost spanning tree**.
   Prims Algorithm, Kruskals Algorithm

8. **Write the difference between the Greedy method and Dynamic programming.**

Greedy method	Dynamic programming
1. Only one sequence of decision is generated.	1. Many number of decisions are generated.
2. It does not guarantee to give an optimal solution always.	2. It definitely gives an optimal solution always.

9. **State the time efficiency of floyd's algorithm**
   $O(n^3)$.It is cubic.

10. **Give the time complexity and space complexity of traveling salesperson problem.**
Time complexity is O (n^2 2^n).
Space complexity is O (n 2^n).

11. **What is the average case complexity for merge sort algorithm?**
Time complexity is O(nlogn)

12. **Quick sort uses _**
Divide and Conquer Technique

13. **What is the best case complexity for insertion sort**
Time complexity is O(n)

14. **Which of the following standard algorithms is not Dynamic Programming based**.
- Bellman–Ford Algorithm for single source shortest path
- Floyd Warshall Algorithm for all pairs shortest paths
- Knapsack problem
- Prim‟s Minimum Spanning Tree
- Prim‟s Minimum Spanning Tree is a Greedy Algorithm. All other are dynamic programming based.

15. **The recurrence relation capturing the optimal time of the Tower of Hanoi problem with n discs is**.
(a) t(n) = 2t(n – 2) +2
(b) t(n) = 2t(n – 1) +n
(c) t(n) = 2t(n/2) + 1
(d) t(n) = 2t(n – 1) + 1

16. **Define spanning tree**
A spanning tree of a connected graph is its connected acyclic subgraph (i.e a tree) that contains all vertices of a graph.

17. **Define Minimum Spanning tree.**
A Minimum Spanning tree of a weighted connected graph is its spanning tree of the smallest weight, where weight of the tree is defined sum of the weights on all its edges.

18. **Define Kruskal's Algorithm**
Kruskal's algorithm looks at a minimum spanning tree for a weighted connected graph G = (V,E) as an acyclic subgraph with | V| - 1 edges for which the sum of the edge weights is the smallest

19. Define Prim's Algorithm

Prim's algorithm is a greedy algorithm for constructing a minimum spanning tree of a weighted connected graph. It works by attaching to a previously constructed subtree a vertex to the vertices already in the tree.

20. Explain Warshalls algorithm

Warshall's algorithm constructs the transitive closure of a given digraph with n vertices through a series of n by n Boolean matrices R(0), ……, R(k-l)R(k),…….., R(n) Each of these matrices provides certain information about directed paths in the digraph.

21. Explain All-pair shortest-paths problem

Given a weighted connected graph (undirected or directed), the all pairs shortest paths problem asks to find the distances(the lengths of the shortest path) from each vertex to all other vertices.

22. Explain Knapsack problem

Given n items of known weights w1,w2.....wn and values v1,v2.....vn and aknapsackofcapacityW,findthemostvaluablesubsetoftheitemsthatfitintotheknapsack.(Assuming all the weights and the knapsack's capacity are positive integers the itemvalues do not have to be integers.)

23. Explain ²Traveling salesman problem”?

A salesman has to travel n cities starting from any one of the cities and visit the remaining cities exactly once and come back to the city where he started his journey in such a manner that either the distance is minimum or cost is minimum. This is known as traveling salesman problem.

24. Explain Backtracking

The principal idea is to construct solutions one component at a time and evaluate such partially constructed candidates as follows.

- If a partially constructed solution can be developed further without violating the problem's constraints, it is done by taking the first remaining legitimate option for the next component.
- If there is no legitimate option for the next component, no alternatives for any remaining component need to be considered. In this case, the algorithm backtracks to replace the last component of the partially constructed solution with its next option

25. Explain promising and non-promising node.

A node in a state space tree is said to be promising if it corresponds to a partially constructed solution that may still lead to a complete solution; otherwise it is called non- promising.

26. Explain N-Queens problem.

The problem is to place N queens on an N by N chessboard so that no two queens attack each other by being in the same row or same column or on the same diagonal.

27. Explain Subset-Sum Problem.

We consider the subset-sum problem: Find a subset of a given set S={S1,S2,.....Sn}of N positive integers whose sum is equal to a given positive integer d.

28. Explain Knapsack Problem.

Find the most valuable subset of n items of given positive integer weights and values that fit into a knapsack of a given positive integer capacity.

29. Define order of growth.

The efficiency analysis framework concentration the order of growth of analgorithm's basic operation count as the principal indicator of the algorithms efficiency. To compare and rank such orders of growth we use three notations

- (Big oh)notation
- Ω (Big Omega) notation&
- Θ (Big Theta)notation

30. What is the use of Asymptotic Notations?

The notations O, Ω and Θ and are used to indicate and compare the asymptotic orders of growth of functions expressing algorithm efficiencies.

CHAPTER 5

Additional Programs

1) Sort a given set of n integer elements using Bubble Sort method and compute its time complexity.

Source Code:

```
#include  <stdio.h>
#include <stdlib.h>
int count=0;

void bubblesort(int A[10],int n)
{
   int i,j,temp;
   for(i=0;i<=n-2;i++)
   {
      for(j=0;j<=n-2-i;j++)
      {
         count++;
         if(A[j] > A[j+1])
         {
            temp=A[j];
            A[j]=A[j+1];
            A[j+1]=temp;
         }
      }
   }

}
int main()
{
   int A[10],n,i,j;
   printf("\n Read the number of elements :");
   scanf("%d",&n);
   printf("\n Read the elements\n");
   for(i=0;i<n;i++)
      scanf("%d",&A[i]);
   bubblesort(A,n);
   printf("\n Elements after sorting\n");
   for(i=0;i<n;i++)
      printf("%d\n",A[i]);
   printf("\n Number of times the basic operation gets executed is %d\n",count);
   return 0;
}
```

2) Sort a given set of n integer elements using Heap Sort method and compute its time complexity.

```
#include<stdio.h>
#include<stdlib.h>
int count;

void heapcon(int a[10],int n)
{
  int i,k,v,flag,j;
  for(i=n/2; i>=1; i--)
  {
    k=i;
    v=a[k];
    flag = 0;
    while ( !flag && (2*k<=n) )
    {
      j=2*k;
      if(j<n)
  if(a[j] < a[j+1])
        {
    j=j+1;

  }
  count++;
    if(v>=a[j])
    flag = 1;
    else
  {
  a[k]=a[j];
        k=j;
  }
    }
    a[k]=v;
  }
}

void heapsort(int a[10], int n)
{
  int i,j,temp;
  for(i=n;i>=1;i--)
  {
    temp=a[1];
    a[1]=a[i];
    a[i]=temp;
    heapcon(a,i-1);
  }
}
```

```
void main()
{
 int a[10],n,i,j;
 printf("\n enter the number of elements to be sorted : ");
 scanf("%d",&n);
 printf("\n Enter the elements to be sorted\n");
 for(i=1;i<=n;i++)
 scanf("%d",&a[i]);
 heapcon(a,n);
 heapsort(a,n);
 printf("\n Elements after sorting\n");
 for(i=1;i<=n;i++)
 printf("%d ",a[i]);
 printf("\n Number of counts : %d\n",count);
 return 0;
}
```

3) Sort a given set of n integer elements using Insertion Sort method and compute its time complexity

```
#include  <stdio.h>
#include <stdlib.h>
int count=0;
void Insertionsort(int A[10],int n)
{
   int i,j,item;
   for(i=1;i<=n-1;i++)
   {
      item= A[i];
      j=i-1;
      while(j>=0 && A[j]>item)
      {
         count++;
         A[j+1]=A[j];
         j=j-1;
      }
      count++;
     A[j+1]=item;
  }

}
int main()
{
   int A[10],n,i,j;
   printf("\n Read the number of elements :");
   scanf("%d",&n);
   printf("\n Read the elements\n");
   for(i=0;i<n;i++)
     scanf("%d",&A[i]);
   Insertionsort(A,n);
   printf("\n Elements after sorting\n");
   for(i=0;i<n;i++)
     printf("%d\n",A[i]);
   printf("\n Number of times the basic operation gets executed is %d\n",count);
   return 0;
}
```

APPENDIX

Time Complexity of Sorting Algorithms

Algorithm Name	Average	Best	Worst	Space	Stability	Remarks
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	Stable	Always use a modified bubble sort
Modified Bubble sort	$O(n^2)$	O(n)	$O(n^2)$	Constant	Stable	Stops after reaching a sorted array
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	InStable	Even a perfectly sorted input requires scanning the entire array
Insertion Sort	$O(n^2)$	O(n)	$O(n^2)$	Constant	Stable	In the best case (already sorted), every insert requires constant time
Heap Sort	O(n*log(n))	O(n*log(n))	O(n*log(n))	Constant	Instable	By using input array as storage for the heap, it is possible to achieve constant space
Merge Sort	O(n*log(n))	O(n*log(n))	O(n*log(n))	Depends	Stable	On arrays, merge sort requires O(n) space; on linked lists, merge sort requires constant space
Quicksort	Ω (n*log(n))	Θ(1.38n*log(n))	$O(n^2)$	Constant	InStable	Randomly picking a pivot value (or shuffling the array prior to sorting) can help avoid worst case scenarios such as a perfectly sorted Array.